Course 5: Foundation Models



Course 5: Foundation Models

Fifth session, and the one that pushes AI/Deep Learning models to their current level.
Session 4 left you with a problem: Deep networks learn their own representation, which is what makes them powerful; but they need an enormous amount of data to do it, which is what makes them disappointing on a thousand samples (like in your project). The last slide of session 4 stated the way out in one line: you will not train such a model, you will *reuse* one.

Today is about the models you reuse (what they are, how they are built, why they work on data nobody prepared for you) and about how you make one of them yours. The same question as in session 4 runs through the session, only one level up: what does this model already know, and how far is your problem from what it has seen?

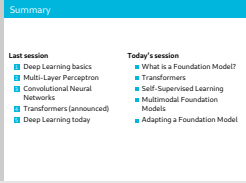
Summary

Last session

1. Deep Learning basics
2. Multi-Layer Perceptron
3. Convolutional Neural Networks
4. Transformers (announced)
5. Deep Learning today

Today's session

- What is a Foundation Model?
- Transformers
- Self-Supervised Learning
- Multimodal Foundation Models
- Adapting a Foundation Model



The first item defines the object. The second and the third are its two ingredients: an architecture that accepts any kind of data (the Transformer) and a way of training on data that nobody annotated (self-supervision).
The fourth item insists on an misunderstood aspect of recent models: they may be **multimodal**, meaning they can process and understand multiple types of data, such as text, images, and audio, at the same time! This may help you in your project.
The fifth item is about the process of adapting a foundation model to a specific task, which involves taking such a model and making it work on *your* problem.

Foundation models **are** deep learning models. But, compared to session 4, which was about architectures and how they are trained from scratch, we now have a pre-trained model that can be fine-tuned for specific tasks. Said differently: today, somebody else has already paid for the training, and the engineering question moves to reuse.

Outline

1 What is a Foundation Model?

2 Transformers

3 Self-supervised Learning

4 Multimodal Foundation Models

5 Adapting a Foundation Model

6 Conclusion

Outline

1 What is a Foundation Model?

2 Transformers

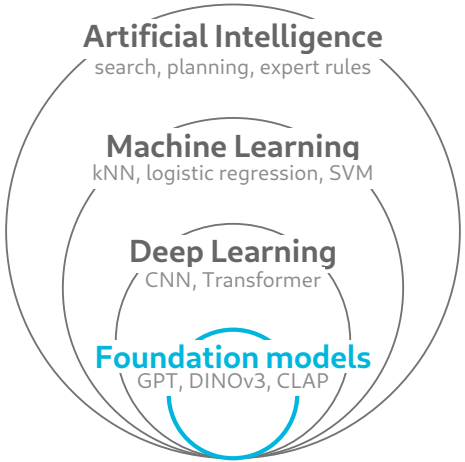
3 Self-supervised Learning

4 Multimodal Foundation Models

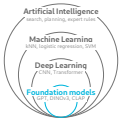
5 Adapting a Foundation Model

6 Conclusion

What is a Foundation Model?



What is a Foundation Model?



The term was coined at Stanford in 2021: a “foundation” model is not the answer to a problem, it is the *floor* on which you build one.
The model was not trained on a single task, instead it will be judged by the quality of the representation it produces. These representations may be probed with the tools of sessions 2 and 3. And **you do not train it**: the pretraining costs millions of euros, so what you actually do is using the model “frozen” (kept as it is), or particularizing it for your specific use case, as shown in the last section of today.

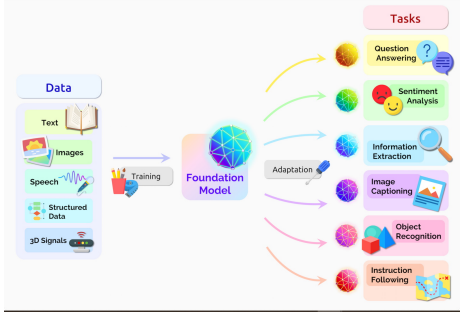
The orange line reverses the message of session 2. There, the difficulty was generalizing from a small training set. Here, a model that has seen a large part of the visible internet already generalizes; the hard part is bringing it down onto *your* narrow problem, with your sensors, your vocabulary, your five classes. That move is called “particularization”, or “adaptation”.

What is a Foundation Model?

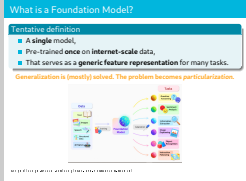
Tentative definition

- A **single** model,
- Pre-trained **once** on **internet-scale** data,
- That serves as a **generic feature representation** for many tasks.

Generalization is (mostly) solved. The problem becomes *particularization*.



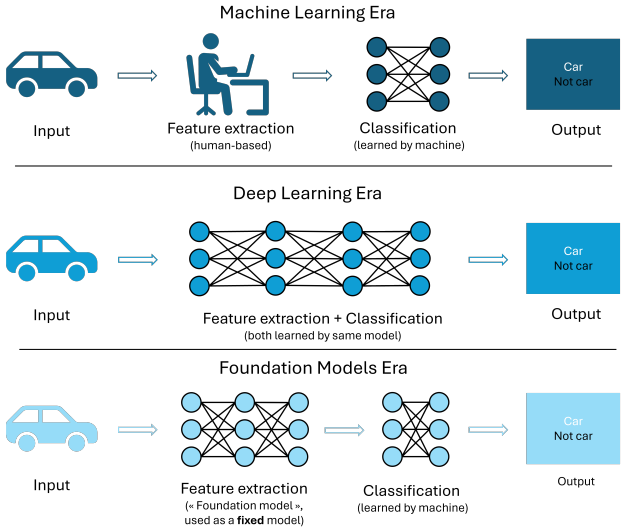
https://blogs.nvidia.com/blog/what-are-foundation-models/

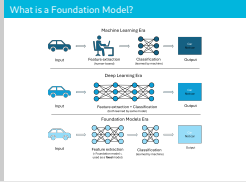


The term was coined at Stanford in 2021: a "foundation" model is not the answer to a problem, it is the *floor* on which you build one.
The model was not trained on a single task, instead it will be judged by the quality of the representation it produces. These representations may be probed with the tools of sessions 2 and 3. And **you do not train it**: the pre-training costs millions of euros, so what you actually do is using the model "frozen" (kept as it is), or particularizing it for your specific use case, as shown in the last section of today.

The orange line reverses the message of session 2. There, the difficulty was generalizing from a small training set. Here, a model that has seen a large part of the visible internet already generalizes; the hard part is bringing it down onto *your* narrow problem, with your sensors, your vocabulary, your five classes. That move is called "particularization", or "adaptation".

What is a Foundation Model?





The term was coined at Stanford in 2021: a “foundation” model is not the answer to a problem, it is the *floor* on which you build one.
The model was not trained on a single task, instead it will be judged by the quality of the representation it produces. These representations may be probed with the tools of sessions 2 and 3. And **you do not train it**: the pre-training costs millions of euros, so what you actually do is using the model “frozen” (kept as it is), or particularizing it for your specific use case, as shown in the last section of today.

The orange line reverses the message of session 2. There, the difficulty was generalizing from a small training set. Here, a model that has seen a large part of the visible internet already generalizes; the hard part is bringing it down onto *your* narrow problem, with your sensors, your vocabulary, your five classes. That move is called “particularization”, or “adaptation”.

Where do Foundation Models come from?

Ingredient 1: Transformer

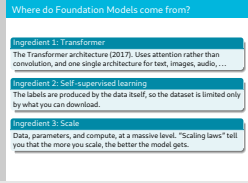
The Transformer architecture (2017). Uses attention rather than convolution, and one single architecture for text, images, audio, …

Ingredient 2: Self-supervised learning

The labels are produced by the data itself, so the dataset is limited only by what you can download.

Ingredient 3: Scale

Data, parameters, and compute, at a massive level. “Scaling laws” tell you that the more you scale, the better the model gets.



Three ingredients for Foundation Models:
Ingredient 1 is a new architecture: Transformers. It assumes only that the input is a set of elements that interact, which is weaker than for CNNs.
Ingredient 2 answers the data problem: ImageNet needed years of human annotation for a million images, and that route does not scale to a billion.
Ingredient 3 is not a detail: the same architecture and the same pretext task, trained on a hundred times less data, gives a model nobody would call a foundation model (ex: from GPT-2 to GPT-3). Transformers have been shown to scale well, and the more you scale, the better they get. The *scaling laws* are empirical, but they are very clear: if you want a foundation model, you need to scale.

Outline

1 What is a Foundation Model?

2 Transformers

3 Self-supervised Learning

4 Multimodal Foundation Models

5 Adapting a Foundation Model

6 Conclusion

Outline
1 What is a Foundation Model?
2 Transformers
3 Self-supervised Learning
4 Multimodal Foundation Models
5 Adapting a Foundation Model
6 Conclusion

Where the Transformer stands

The standard architecture today

- **No convolution**, no fixed neighbourhood,
- Based on *attention*: **which** elements matter for **which**?
- Used for text, images, audio, video, protein sequences, …

what does the architecture assume about the data? (session 4)

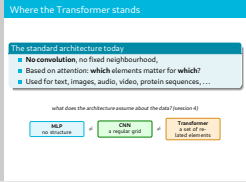
MLP
no structure

≠

CNN
a regular grid

≠

Transformer
a set of related elements



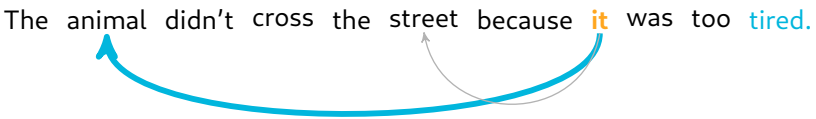
A few words (again) about the different architectures.
An MLP assumes nothing about the input, and pays for it in parameters. A CNN is *given* locality and translation, which is knowledge you do not have to learn, but mostly adapted for Images. That is why it wins on images, even with with modest datasets. A Transformer assumes only that the input is a collection of elements that interact, without saying which ones interact; it **learns** the neighbourhood instead of receiving it.

Both sides of that trade matter. Because the assumption is weak, the same architecture runs on a sentence, a photograph and a spectrogram, which is exactly what a general-purpose model needs. And because the assumption is weak, the model has to discover from examples what a CNN was told for free, so it only becomes better than a CNN when the data is abundant. Foundation models are what happens when the data *is* abundant.

The intuition behind attention

Attention asks one question, for each element

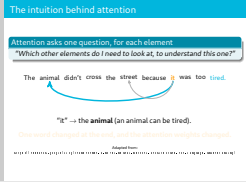
"Which other elements do I need to look at, to understand this one?"



"it" → the **animal** (an animal can be tired).

One word changed at the end, and the attention weights changed.

Adapted from:
https://research.google/blog/transformer-a-novel-neural-network-architecture-for-language-understanding/



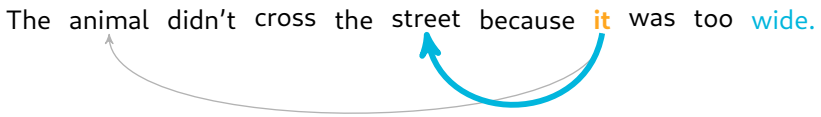
A classical example, to make attention concrete without a single equation. Answer the question yourself before reading on: in the first sentence, what does "it" refer to?
It's obviously "the animal". Change one word at the very end of the sentence and the answer becomes "the street". This is a major problem for text processing, that attention handles very well.
The point is that this dependency is **not positional**. "animal" is six words away, "street" two, and the correct answer switches from one to the other because of an adjective placed at the end. A convolution, with its fixed window, cannot express that; and a window wide enough to reach "animal" would drown it in six irrelevant words. Attention computes, for every pair of elements, a weight saying how much one matters for the other (the arrow thicknesses in the drawing), and those weights are recomputed for every new input.

Note also what the arrows are *for*. The goal is not to draw pretty links: it is to build a representation of "it" that already contains "animal". A word does not have one vector, it has one vector *per context*.

The intuition behind attention

Attention asks one question, for each element

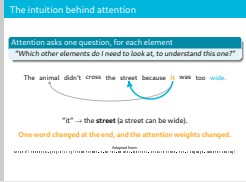
"Which other elements do I need to look at, to understand this one?"



"it" → the **street** (a street can be wide).

One word changed at the end, and the attention weights changed.

Adapted from:
https://research.google/blog/transformer-a-novel-neural-network-architecture-for-language-understanding/



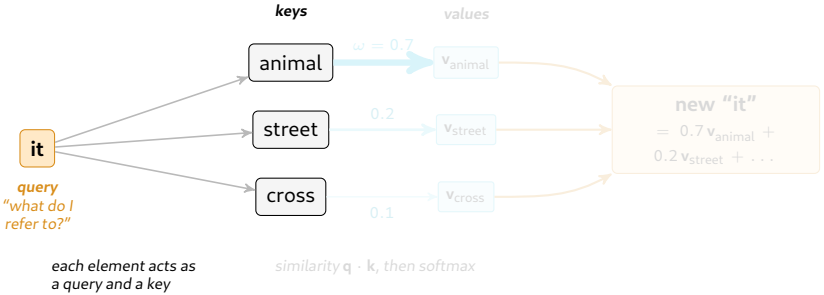
A classical example, to make attention concrete without a single equation. Answer the question yourself before reading on: in the first sentence, what does "it" refer to?
It's obviously "the animal". Change one word at the very end of the sentence and the answer becomes "the street". This is a major problem for text processing, that attention handles very well.
The point is that this dependency is **not positional**. "animal" is six words away, "street" two, and the correct answer switches from one to the other because of an adjective placed at the end. A convolution, with its fixed window, cannot express that; and a window wide enough to reach "animal" would drown it in six irrelevant words. Attention computes, for every pair of elements, a weight saying how much one matters for the other (the arrow thicknesses in the drawing), and those weights are recomputed for every new input.

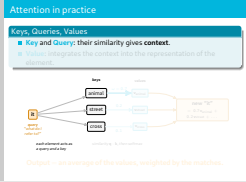
Note also what the arrows are *for*. The goal is not to draw pretty links: it is to build a representation of "it" that already contains "animal". A word does not have one vector, it has one vector *per context*.

Attention in practice

Keys, Queries, Values

- **Key** and **Query**: their similarity gives **context**.
- **Value**: integrates the context into the representation of the element.





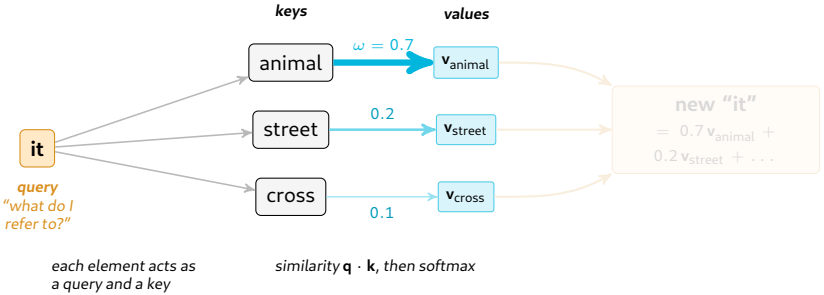
Query, Key Value: the three magic words of attention. The essential idea: attention is a dictionary look-up that is allowed to be fuzzy.
Read the block as two steps, because they answer two different questions. **Where to look** is settled by the queries and the keys. The similarity is a scalar product, and the scores become weights summing to one through a softmax: that set of weights *is* the context of the sentence.
How to integrate the context is settled by the values: instead of returning one entry, the mechanism returns a **mixture** of the values, in those proportions. So the output is a *new* vector for "it", which now contains "animal". With attention, a word does not have one vector, it has one vector *per context*, and that is the most important consequence of the mechanism.

A note, for the vocabulary: when the keys and queries come from the same input it is *self*-attention. When queries come from one input and keys from another it is *cross*-attention, which is how attention was invented in 2014, for translation.

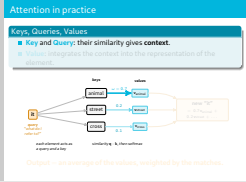
Attention in practice

Keys, Queries, Values

- **Key** and **Query**: their similarity gives **context**.
- **Value**: integrates the context into the representation of the element.



Output = an average of the values, weighted by the matches.



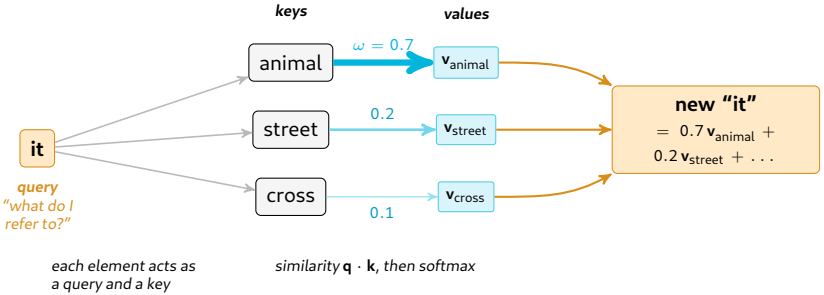
Query, Key Value: the three magic words of attention. The essential idea: attention is a dictionary look-up that is allowed to be fuzzy.
Read the block as two steps, because they answer two different questions. **Where to look** is settled by the queries and the keys. The similarity is a scalar product, and the scores become weights summing to one through a softmax: that set of weights *is* the context of the sentence.
How to integrate the context is settled by the values: instead of returning one entry, the mechanism returns a **mixture** of the values, in those proportions.
So the output is a *new* vector for "it", which now contains "animal". With attention, a word does not have one vector, it has one vector *per context*, and that is the most important consequence of the mechanism.

A note, for the vocabulary: when the keys and queries come from the same input it is *self*-attention. When queries come from one input and keys from another it is *cross*-attention, which is how attention was invented in 2014, for translation.

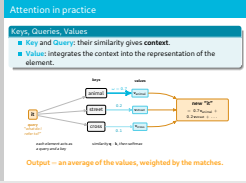
Attention in practice

Keys, Queries, Values

- **Key** and **Query**: their similarity gives **context**.
- **Value**: integrates the context into the representation of the element.



Output = an average of the values, weighted by the matches.



Query, Key Value: the three magic words of attention. The essential idea: attention is a dictionary look-up that is allowed to be fuzzy.

Read the block as two steps, because they answer two different questions. **Where to look** is settled by the queries and the keys. The similarity is a scalar product, and the scores become weights summing to one through a softmax: that set of weights *is* the context of the sentence.

How to integrate the context is settled by the values: instead of returning one entry, the mechanism returns a **mixture** of the values, in those proportions. So the output is a *new* vector for "it", which now contains "animal". With attention, a word does not have one vector, it has one vector *per context*, and that is the most important consequence of the mechanism.

A note, for the vocabulary: when the keys and queries come from the same input it is *self*-attention. When queries come from one input and keys from another it is *cross*-attention, which is how attention was invented in 2014, for translation.

Self-attention, formally (1/2): one element at a time

From the drawing to the maths

- Each input $\mathbf{x}_i$ is projected by three **learned** matrices:

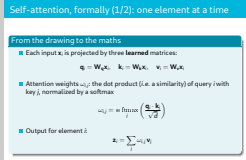
$$\mathbf{q}_i = \mathbf{W_q}\mathbf{x}_i, \quad \mathbf{k}_i = \mathbf{W_k}\mathbf{x}_i, \quad \mathbf{v}_i = \mathbf{W_v}\mathbf{x}_i$$

- Attention weights $\omega_{i,j}$: the dot product (*i.e.* a similarity) of query i with key j, normalized by a softmax

$$\omega_{i,j} = \underset{j}{\mathrm{softmax}}\left(\frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d}}\right)$$

- Output for element i:

$$\mathbf{z}_i = \sum_j \omega_{i,j}\,\mathbf{v}_j$$



The previous slide, more formally in mathematical terms.
The three projections are the **only** trainable objects in the attention mechanism. Thus, a Transformer is trained exactly like any network seen in session 4 (with the exception of scale, that requires specific tricks; this is very out-of-scope here).
Note that the same three matrices are applied to *every* element, which is the weight sharing of the convolutions of session 4, in another guise (it will be clearer in the next slide).
The second formula is the arrow thickness of the drawing. The dot product $\mathbf{q}_i \cdot \mathbf{k}_j$ measures how well element j answers the need of element i. The division by $\sqrt{d}$ (d is the dimension of the vectors) only keeps those numbers in a range where the softmax does not saturate. The softmax then makes the weights positive and summing to one, which is what turns them into proportions.
The third formula is the aggregation: the new representation of element i is the average of the values of all the elements, weighted by how much each one matters to it.

Self-attention, formally (2/2): the whole input at once

Stack the elements into matrices

- Input **X** holds all elements, and $\mathbf{Q} = \mathbf{X}\mathbf{W_q}$, $\mathbf{K} = \mathbf{X}\mathbf{W_k}$, $\mathbf{V} = \mathbf{X}\mathbf{W_v}$,
- Output **Z** is an **attention-weighted version of the input**, w.r.t. all other elements of the input.

$$\mathbf{Z} = \underbrace{\mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)}_{\text{all the } \omega_{i,j} \text{ at once}} \mathbf{V}$$

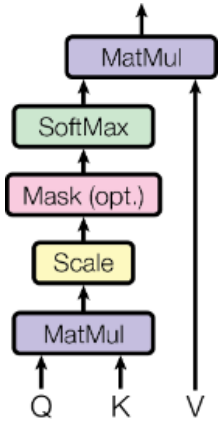
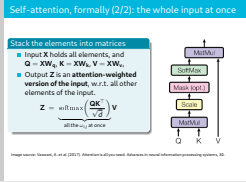


Image source: Vaswani, A. *et al.* (2017). Attention is all you need. Advances in neural information processing systems, 30.



The same operation written once for the whole input, which is how it is actually computed (one matrix multiplication instead of several ones).
Read the equation as the previous slide, done in parallel. $\mathbf{Q}\mathbf{K}^\top$ is the table of *all* the scalar products between every query and every key, so it is a square matrix holding the score of every pair. The softmax is applied row by row, producing all the $\omega_{i,j}$ at once, and multiplying by **V** performs the weighted average for every element. One matrix product instead of a loop, which is what makes it fast on a GPU.
One practical consequence: that square table has one entry per *pair* of elements, so doubling the length of the input multiplies the cost by four. This is why the size of the "context window" is a headline feature of language models, and why a good part of current research is about computing this more cheaply.
Notice, finally, that nothing in the equation mentions the *order* of the elements: self-attention would give the same answer on a shuffled sentence. The position therefore has to be added to the input explicitly, which is called a positional encoding (not represented here).

Repeat, and repeat: the Transformer block

Several heads in parallel: **multi-head** attention

One attention = **one** kind of relation.
Several heads: several concepts (who does what, what refers to what, which words agree, …).

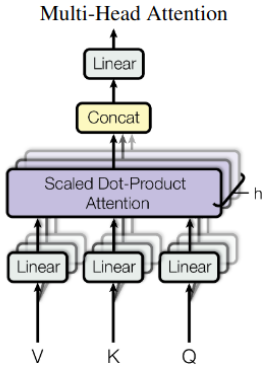
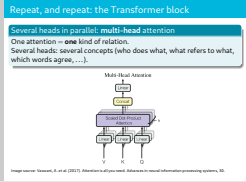


Image source: Vaswani, A. *et al.* (2017). Attention is all you need. Advances in neural information processing systems, 30.



Two mechanical points, and then we are done with the architecture.
Multi-head. One attention produces one set of weights, that is, one way of relating the elements to each other. That is restrictive: in a sentence one may want to track the subject, the referent of a pronoun and the agreement of a verb at once. So the operation is run several times in parallel, with different projection matrices, and the results are concatenated. Each copy is a **head**.
Depth. A block is one multi-head attention followed by a small MLP applied to each token (plus a residual connection and a normalization, and many potential additional details). Blocks are then stacked, and their number is one of the two figures describing a model, the other being the width.

What is worth keeping is the shape of the thing: one operation, repeated in parallel (heads) and in series (depth), with nothing specific to language.

Repeat, and repeat: the Transformer block

Stack the blocks: a **deep** model

Attention + a small MLP = one **block**. Repeat it N times (12, 32, ...)

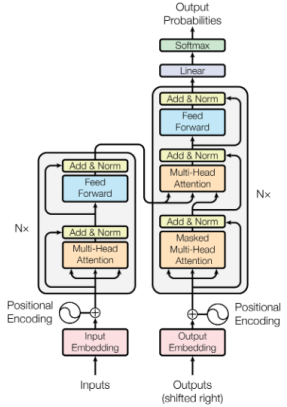
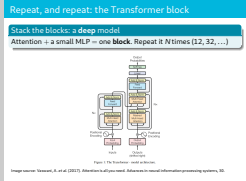


Figure 1: The Transformer - model architecture.

Image source: Vaswani, A. *et al.* (2017). Attention is all you need. Advances in neural information processing systems, 30.



Two mechanical points, and then we are done with the architecture.

Multi-head. One attention produces one set of weights, that is, one way of relating the elements to each other. That is restrictive: in a sentence one may want to track the subject, the referent of a pronoun and the agreement of a verb at once. So the operation is run several times in parallel, with different projection matrices, and the results are concatenated. Each copy is a **head**.

Depth. A block is one multi-head attention followed by a small MLP applied to each token (plus a residual connection and a normalization, and many potential additional details). Blocks are then stacked, and their number is one of the two figures describing a model, the other being the width.

What is worth keeping is the shape of the thing: one operation, repeated in parallel (heads) and in series (depth), with nothing specific to language.

Outline

1 What is a Foundation Model?

2 Transformers

3 Self-supervised Learning

4 Multimodal Foundation Models

5 Adapting a Foundation Model

6 Conclusion

Outline
What is a Foundation Model?
Transformers
Self-supervised Learning
Multimodal Foundation Models
Adapting a Foundation Model
Conclusion

Self-Supervised Learning

The intuition

Corrupt a part of the data, and train the network to reconstruct it.

⇒ the label is **produced by the data itself**: No annotator.

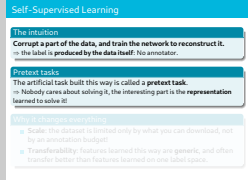
Pretext tasks

The artificial task built this way is called a **pretext task**.

⇒ Nobody cares about solving it, the interesting part is the **representation** learned to solve it!

Why it changes everything

- **Scale**: the dataset is limited only by what you can download, not by an annotation budget!
- **Transferability**: features learned this way are **generic**, and often transfer better than features learned on one label space.



Self-supervision was named in session 1 but never seen again. It may be seen as an intermediate between supervised and unsupervised learning: it is a way to generate supervision from unlabelled data. Yet, it's neither of the two.

The recipe deserves a careful reading, because the trick is almost too simple to notice: we *fabricate* a supervised problem out of unlabelled data. Removing a word from a sentence gives an input (the sentence with a hole) and a ground truth (the missing word) for free, and the machinery of session 4 then applies unchanged. Nothing new in the optimization; the trick is in where the labels come from.

"Pretext" is the important word. The model that predicts the missing word is not the product. The product is what it had to build in order to succeed, i.e. the representations learned in the training process.

It's more broad than supervised learning: A model trained on a thousand classes may ignore everything that does not separate them; a model that must rebuild any missing piece of any image cannot ignore anything, as long as it sees enough images to generalize.

Self-Supervised Learning

The intuition

Corrupt a part of the data, and train the network to reconstruct it.

$\Rightarrow$ the label is **produced by the data itself**: No annotator.

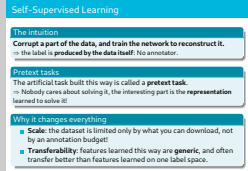
Pretext tasks

The artificial task built this way is called a **pretext task**.

$\Rightarrow$ Nobody cares about solving it, the interesting part is the **representation** learned to solve it!

Why it changes everything

- **Scale**: the dataset is limited only by what you can download, not by an annotation budget!
- **Transferability**: features learned this way are **generic**, and often transfer better than features learned on one label space.



Self-supervision was named in session 1 but never seen again. It may be seen as an intermediate between supervised and unsupervised learning: it is a way to generate supervision from unlabelled data. Yet, it's neither of the two.

The recipe deserves a careful reading, because the trick is almost too simple to notice: we *fabricate* a supervised problem out of unlabelled data. Removing a word from a sentence gives an input (the sentence with a hole) and a ground truth (the missing word) for free, and the machinery of session 4 then applies unchanged. Nothing new in the optimization; the trick is in where the labels come from.

"Pretext" is the important word. The model that predicts the missing word is not the product. The product is what it had to build in order to succeed, i.e. the representations learned in the training process.

It's more broad than supervised learning: A model trained on a thousand classes may ignore everything that does not separate them; a model that must rebuild any missing piece of any image cannot ignore anything, as long as it sees enough images to generalize.

Examples of pretext tasks

1. Predict the hidden part
mask a piece, guess it
BERT, GPT, MAE

2. Compare examples
same thing close, different things apart
SimCLR, CLIP

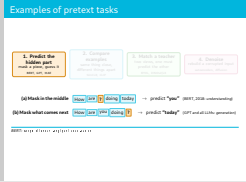
3. Match a teacher
two views, one must predict the other
BYOL, DINOv2/v3

4. Denoise
rebuild a corrupted input
autoencoders, diffusion

(a) Mask in the middle How are **?** doing today → predict **"you"** (BERT, 2018: *understanding*)

(b) Mask what comes next How are you doing **?** → predict **"today"** (GPT and all LLMs: *generation*)

BERT: https://arxiv.org/pdf/1810.04805



A legitimate question after the previous slide: how to define pretext task? We regrouped them as 4 families:

Family 1 hides part of the input and predicts it. This is the most common, in particular in LLMs.

Family 2 predicts nothing, it compares: two views of one image must end up close (ex: a rotated version), two different images far apart.

Family 3 is used when an existing model already exists, and we want to improve or replicate it (for instance a smaller model doing the same thing). Two views are given to two copies of the network, and the "student" is trained to predict the output of the "teacher", itself a slowly moving average of the student. DINOv2 and DINOv3 are trained this way.

Family 4 adds noise to the input, learn how to denoise/rebuild it. Adding *noise* and learning to remove it, step by step, is what a diffusion model does.

Examples of pretext tasks

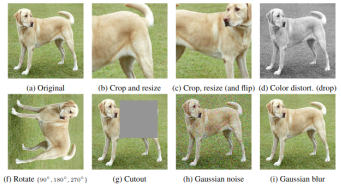
1. **Predict the hidden part**
mask a piece, guess it
BERT, GPT, MAE

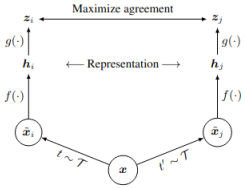
2. **Compare examples**
same thing close, different things apart
SimCLR, CLIP

3. **Match a teacher**
two views, one must predict the other
BYOL, DINOv2/v3

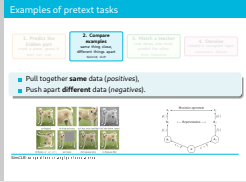
4. **Denoise**
rebuild a corrupted input
autoencoders, diffusion

- Pull together **same** data (*positives*),
- Push apart **different** data (*negatives*).





SimCLR: `https://arxiv.org/pdf/2002.05709`



A legitimate question after the previous slide: how to define pretext task? We regrouped them as 4 families:

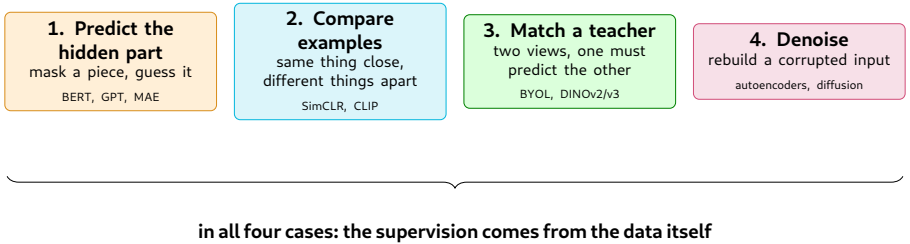
Family 1 hides part of the input and predicts it. This is the most common, in particular in LLMs.

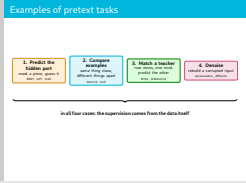
Family 2 predicts nothing, it compares: two views of one image must end up close (ex: a rotated version), two different images far apart.

Family 3 is used when an existing model already exists, and we want to improve or replicate it (for instance a smaller model doing the same thing). Two views are given to two copies of the network, and the "student" is trained to predict the output of the "teacher", itself a slowly moving average of the student. DINOv2 and DINOv3 are trained this way.

Family 4 adds noise to the input, learn how to denoise/rebuild it. Adding *noise* and learning to remove it, step by step, is what a diffusion model does.

Examples of pretext tasks





A legitimate question after the previous slide: how to define pretext task? We regrouped them as 4 families:

Family 1 hides part of the input and predicts it. This is the most common, in particular in LLMs.

Family 2 predicts nothing, it compares: two views of one image must end up close (ex: a rotated version), two different images far apart.

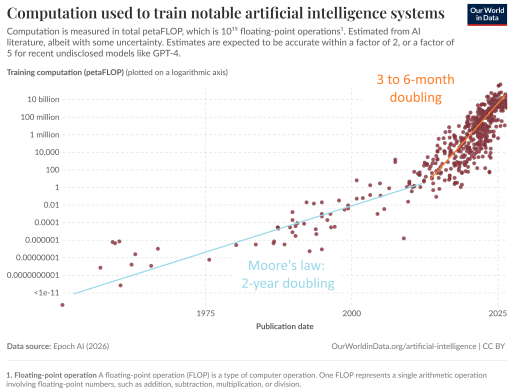
Family 3 is used when an existing model already exists, and we want to improve or replicate it (for instance a smaller model doing the same thing). Two views are given to two copies of the network, and the "student" is trained to predict the output of the "teacher", itself a slowly moving average of the student. DINOv2 and DINOv3 are trained this way.

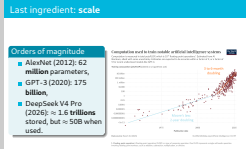
Family 4 adds noise to the input, learn how to denoise/rebuild it. Adding *noise* and learning to remove it, step by step, is what a diffusion model does.

Last ingredient: **scale**

Orders of magnitude

- AlexNet (2012): 62 **million** parameters,
- GPT-3 (2020): 175 **billion**,
- DeepSeek V4 Pro (2026): $\approx$ 1.6 **trillions** stored, but $\approx$ 50B when used.





You have seen this figure twice, in session 1 as a curiosity and in session 4 as a warning. Third time, as a location: this is *where* the models of today sit.
Read the axes carefully. The vertical scale is logarithmic, so each step up is a multiplication and a straight portion is an exponential. Two regimes are visible: a slow one following Moore's law, and, from about 2012, a much steeper one obtained by adding hardware in parallel rather than by making it faster.
Note the caveat in the presentation of the most rcent model (DeepSeek V4 Pro): the model has a trillion parameters, but does not necessarily use them all for a given input. Modern models are often "mixtures of experts", storing very many weights and activating only a small fraction per token, because compute has become the binding constraint.

Keep in mind what this means for you: one such training run costs millions of euros and thousands of tonnes of CO_2. That is the practical reason why the rest of the session is about *reusing*.

Outline

1 What is a Foundation Model?

2 Transformers

3 Self-supervised Learning

4 Multimodal Foundation Models

5 Adapting a Foundation Model

6 Conclusion

Outline
What is a Foundation Model?
Transformers
Self-supervised Learning
Multimodal Foundation Models
Adapting a Foundation Model
Conclusion

What is a Multimodal Foundation Model?

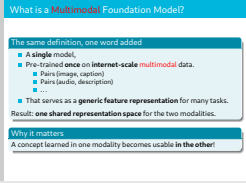
The same definition, one word added

- A **single** model,
- Pre-trained **once** on **internet-scale** multimodal data.
  - Pairs (image, caption)
  - Pairs (audio, description)
  - …
- That serves as a **generic feature representation** for many tasks.

Result: **one shared representation space** for the two modalities.

Why it matters

A concept learned in one modality becomes usable **in the other**!



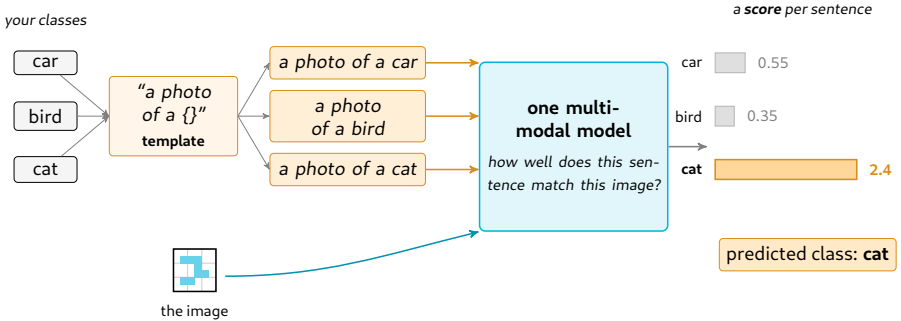
Models nowadays are not restricted to one modality (text, image, audio, video, …): they can be trained on several at once.
The data is now made of **pairs** across modalities. A good property is that the web is full of images that come with a caption, an alt text, or a surrounding paragraph; the same holds for audio with its description, or video with its subtitles. So we are still in the self-supervised regime, at the scale of hundreds of millions of pairs, with no annotator anywhere.

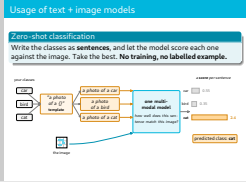
Multimodality may offer new perspectives on a lot of existing tasks, and open new horizons. Take a moment to think about what mulitmodality could offer in the context of your project?

Usage of text + image models

Zero-shot classification

Write the classes as **sentences**, and let the model score each one against the image. Take the best. **No training, no labelled example.**





Two overlays, three things that were out of reach before these models existed. **Zero-shot classification** changes what a classifier is. In sessions 2 and 4, a classifier was trained on labelled examples of *your* classes, and adding a class meant retraining. Here the classes are written as *sentences*, the model scores each sentence against the image, and the best one wins. Adding a class costs one sentence, not one annotation campaign. Note the detail on the figure, which is not a detail in practice: the **template** matters, and "a photo of a {}" works better than the bare word, because it looks more like the captions the model was trained on. This is prompting, in its most primitive form.

Retrieval is the same computation read in the other direction, and it is what an image search engine does today.

Text to image generation is a more recent development, and it is the one that has made the most noise in the press (and you have probably already played with it yourself).

Usage of text + image models

New tasks, beyond classification and segmentation

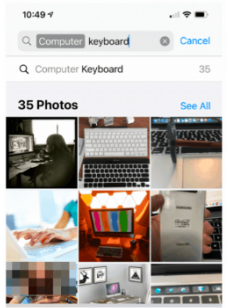
Multimodal models allow to go beyond standard tasks, and to do things that were out of reach before!

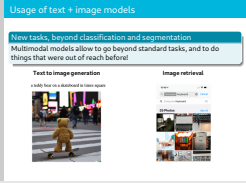
Text to image generation

a teddy bear on a skateboard in times square



Image retrieval





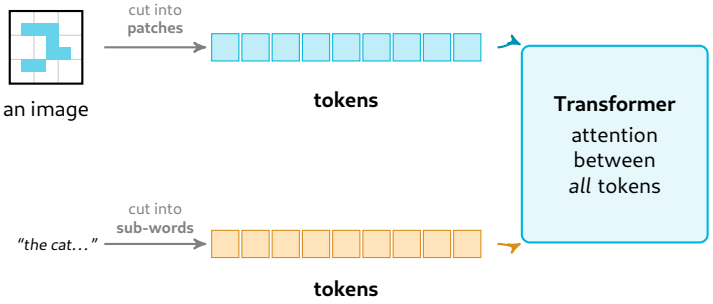
Two overlays, three things that were out of reach before these models existed. **Zero-shot classification** changes what a classifier is. In sessions 2 and 4, a classifier was trained on labelled examples of *your* classes, and adding a class meant retraining. Here the classes are written as *sentences*, the model scores each sentence against the image, and the best one wins. Adding a class costs one sentence, not one annotation campaign. Note the detail on the figure, which is not a detail in practice: the **template** matters, and "a photo of a {}" works better than the bare word, because it looks more like the captions the model was trained on. This is prompting, in its most primitive form.
Retrieval is the same computation read in the other direction, and it is what an image search engine does today.

Text to image generation is a more recent development, and it is the one that has made the most noise in the press (and you have probably already played with it yourself).

How to get multimodality (1/2)? One Transformer

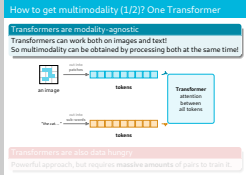
Transformers are modality-agnostic

Transformers can work both on images and text!
So multimodality can be obtained by processing both at the same time!



Transformers are also data hungry

Powerful approach, but requires **massive amounts** of pairs to train it.



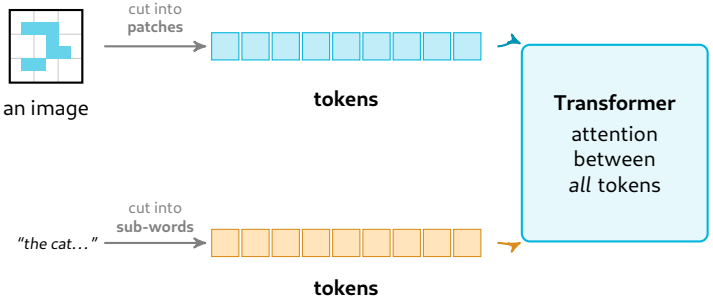
This slide answers both "how to develop mutlimodal models?" and "how do we get from images to text with the same architecture?".
The answer is that a Transformer does not process images nor does it process text. It processes a *sequence of vectors*, called **tokens**, and computes attention between all pairs of them. So the only thing to build for a new modality is a converter into tokens.
For text: cut into sub-words and look up a vector for each (the "tokenizer").
For an image: cut into patches of 16×16 pixels and project each one, which is the Vision Transformer (2020).
For audio: slice the waveform, or the spectrogram.
Then, the multimodality naturally follows: since the input is only a sequence of vectors, **nothing prevents mixing** tokens of different origins in the *same* sequence.

Nonetheless, the approach is very data hungry: the model must learn to relate the two modalities while it must also learn how to process each modality individually.

How to get multimodality (1/2)? One Transformer

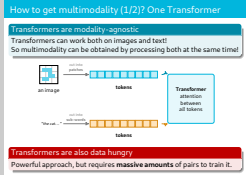
Transformers are modality-agnostic

Transformers can work both on images and text!
So multimodality can be obtained by processing both at the same time!



Transformers are also data hungry

Powerful approach, but requires **massive amounts** of pairs to train it.



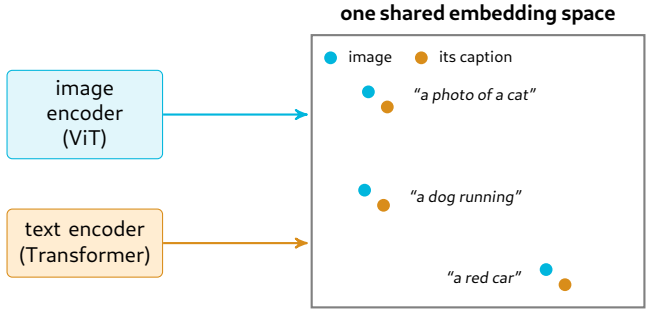
This slide answers both "how to develop mutlimodal models?" and "how do we get from images to text with the same architecture?".
The answer is that a Transformer does not process images nor does it process text. It processes a *sequence of vectors*, called **tokens**, and computes attention between all pairs of them. So the only thing to build for a new modality is a converter into tokens.
For text: cut into sub-words and look up a vector for each (the "tokenizer").
For an image: cut into patches of 16×16 pixels and project each one, which is the Vision Transformer (2020).
For audio: slice the waveform, or the spectrogram.
Then, the multimodality naturally follows: since the input is only a sequence of vectors, **nothing prevents mixing** tokens of different origins in the *same* sequence.

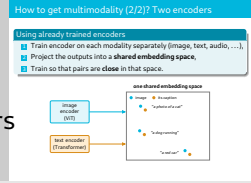
Nonetheless, the approach is very data hungry: the model must learn to relate the two modalities while it must also learn how to process each modality individually.

How to get multimodality (2/2)? Two encoders

Using already trained encoders

1. Train encoder on each modality separately (image, text, audio, …),
2. Project the outputs into a **shared embedding space**,
3. Train so that pairs are **close** in that space.





Multimodality is also possible using existing models, and thus models trained on specific modalities.
The mechanism is simple: use one encoder per modality, and project their outputs into a common space, trained so that an image and its caption land at the **same place** in a common space. Note that the two encoders remain completely separate and never see each other's input; the only thing tying them together is the projection loss.
Examples of tasks that can be reached with such models:
Search: encode the sentence, return the nearest images.
Zero-shot classification: encode "a photo of a cat" and "a photo of a dog", and give the image the closest label.

Generation: take the point of a sentence, and ask a decoder for an image there.

Multimodal models: a quick tour of what's possible

T Text I Image A Audio V Video

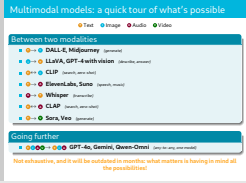
Between two modalities

- T → I **DALL·E, Midjourney** *(generate)*
- I → T **LLaVA, GPT-4 with vision** *(describe, answer)*
- T ↔ I **CLIP** *(search, zero-shot)*
- T → A **ElevenLabs, Suno** *(speech, music)*
- A → T **Whisper** *(transcribe)*
- T ↔ A **CLAP** *(search, zero-shot)*
- T → V **Sora, Veo** *(generate)*

Going further

- T I A V → T I A **GPT-4o, Gemini, Qwen-Omni** *(any-to-any, one model)*

Not exhaustive, and it will be outdated in months: what matters is having in mind all the possibilities!



A deliberately dense slide, built to be scanned rather than read line by line: the badges say which modality goes in and which comes out, so you do not need to parse a sentence to place a model you have heard of.
While multimodal is first thought as text and image, it is way beyond that: audio and video may also be handled. Very recent models are even any-modal: you can give them text, image, audio, or video, and they will answer in any of the four. The only limit is what they have been trained on.
Whatever is your modality, there is probably a model that can handle it!

The useful skill to learn from this slide: given a new model name next month, being able to place it on this kind of map (what goes in, what comes out) in a few seconds.

Outline

1 What is a Foundation Model?

2 Transformers

3 Self-supervised Learning

4 Multimodal Foundation Models

5 Adapting a Foundation Model

6 Conclusion

Outline
What is a Foundation Model?
Transformers
Self-supervised Learning
Multimodal Foundation Models
Adapting a Foundation Model
Conclusion

Adapting a model?

The situation

Foundation models extract **general-purpose** representations and understand your data at a high level.

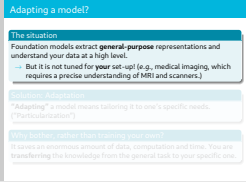
→ But it is not tuned for **your** set-up! (*e.g.*, medical imaging, which requires a precise understanding of MRI and scanners.)

Solution: Adaptation

"Adapting" a model means tailoring it to one's specific needs. ("Particularization")

Why bother, rather than training your own?

It saves an enormous amount of data, computation and time. You are **transferring** the knowledge from the general task to your specific one.



Problem definition: A foundation model knows a great deal in general and nothing in particular. It has an excellent idea of what an edge, a texture or a scene is; it has never seen *your* MRI machine, *your* five defect categories, or the vocabulary of *your* field. Adaptation closes the gap between "understands images" and "answers my question about my images".

Solution: Adaptation is the process of tailoring a pre-trained model to a specific task or domain. It involves adjusting the model's parameters to better fit the characteristics of the new data. We will present three techniques: Transfer Learning, Fine-tuning, and Parameter-Efficient Fine-Tuning (PEFT).

Once again, the key point is that adaptation is **much cheaper** than training a model from scratch. Pre-training is a one-time, resource-intensive process, while adaptation can be done quickly and efficiently on your own data and hardware. (We insist on this point; it should be almost too repetitive by now!)

Adapting a model?

The situation

Foundation models extract **general-purpose** representations and understand your data at a high level.

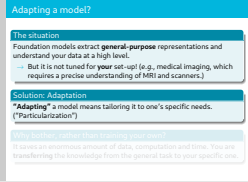
→ But it is not tuned for **your** set-up! (*e.g.*, medical imaging, which requires a precise understanding of MRI and scanners.)

Solution: Adaptation

"Adapting" a model means tailoring it to one's specific needs. ("Particularization")

Why bother, rather than training your own?

It saves an enormous amount of data, computation and time. You are **transferring** the knowledge from the general task to your specific one.



Problem definition: A foundation model knows a great deal in general and nothing in particular. It has an excellent idea of what an edge, a texture or a scene is; it has never seen *your* MRI machine, *your* five defect categories, or the vocabulary of *your* field. Adaptation closes the gap between "understands images" and "answers my question about my images".

Solution: Adaptation is the process of tailoring a pre-trained model to a specific task or domain. It involves adjusting the model's parameters to better fit the characteristics of the new data. We will present three techniques: Transfer Learning, Fine-tuning, and Parameter-Efficient Fine-Tuning (PEFT).

Once again, the key point is that adaptation is **much cheaper** than training a model from scratch. Pre-training is a one-time, resource-intensive process, while adaptation can be done quickly and efficiently on your own data and hardware. (We insist on this point; it should be almost too repetitive by now!)

Adapting a model?

The situation

Foundation models extract **general-purpose** representations and understand your data at a high level.

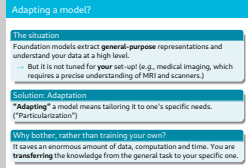
→ But it is not tuned for **your** set-up! (*e.g.*, medical imaging, which requires a precise understanding of MRI and scanners.)

Solution: Adaptation

"Adapting" a model means tailoring it to one's specific needs. ("Particularization")

Why bother, rather than training your own?

It saves an enormous amount of data, computation and time. You are **transferring** the knowledge from the general task to your specific one.



Problem definition: A foundation model knows a great deal in general and nothing in particular. It has an excellent idea of what an edge, a texture or a scene is; it has never seen *your* MRI machine, *your* five defect categories, or the vocabulary of *your* field. Adaptation closes the gap between "understands images" and "answers my question about my images".

Solution: Adaptation is the process of tailoring a pre-trained model to a specific task or domain. It involves adjusting the model's parameters to better fit the characteristics of the new data. We will present three techniques: Transfer Learning, Fine-tuning, and Parameter-Efficient Fine-Tuning (PEFT).

Once again, the key point is that adaptation is **much cheaper** than training a model from scratch. Pre-training is a one-time, resource-intensive process, while adaptation can be done quickly and efficiently on your own data and hardware. (We insist on this point; it should be almost too repetitive by now!)

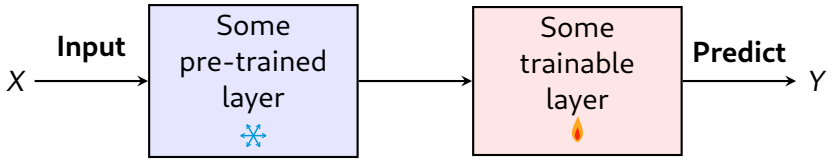
Notations

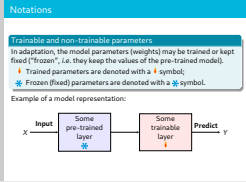
Trainable and non-trainable parameters

In adaptation, the model parameters (weights) may be trained or kept fixed (“frozen”, *i.e.* they keep the values of the pre-trained model).

- Trained parameters are denoted with a symbol;
- Frozen (fixed) parameters are denoted with a symbol.

Example of a model representation:





A small notation slide, because the next diagrams all use it and because the distinction it draws is the only real choice in adaptation.

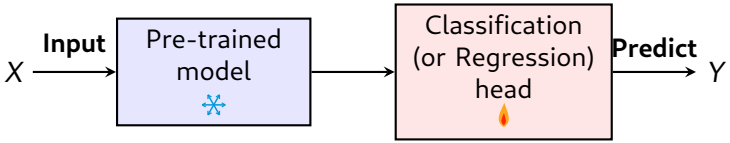
A parameter is either **frozen**, meaning it keeps the value it had after pre-training and receives no gradient, or **trained**, typically by the gradient descent seen in session 4. Nothing else. Every technique in this section is a different answer to a single question: which parameters do I put in which box, and do I add any new ones?

Two practical consequences of freezing, worth knowing before the lab. A frozen part still has to be *run* forward, so it costs computation at every step; what you save is the backward pass, the storage of the gradients, and the optimizer state, which together are the bulk of the memory. And in PyTorch this is one line (`requires_grad = False`), plus the discipline of giving the optimizer only the parameters you actually want to move.

Transfer Learning (linear probing)

Tentative definition

- Pre-trained model: **fixed feature extractor**,
- Add one (or several) layers on top of the representation.



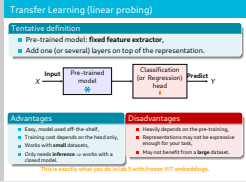
Advantages

- Easy, model used off-the-shelf,
- Training cost depends on the head only,
- Works with **small** datasets,
- Only needs **inference** ⇒ works with a closed model.

Disadvantages

- Heavily depends on the pre-training,
- Representations may not be expressive enough for your task,
- May not benefit from a **large** dataset.

This is exactly what you do in lab 5 with frozen ViT embeddings.



The cheapest form of reuse, the one you are doing in the lab, and the one you should always try first.

The mechanism has no surprise in it: run your data through the frozen model once, keep the vectors it produces, and treat them as the feature matrix for your data. A logistic regression or a k-NN on top, and you are done. Note that the backbone is used *once* per sample, so you can compute the embeddings, save them to disk, and then iterate on classifiers at no cost.

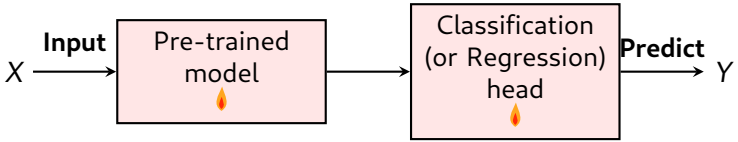
The last advantage is easy to overlook and often decisive in industry: this method only requires the ability to *call* the model, so it works with a closed, API-only one. Every other technique here needs the weights.

The third disadvantage is the subtle one. Since the representation is fixed, more data can only help the small head, so the curve flattens quickly. Observing that flattening while you still have data left is your signal to move on to more complex methods.

Fine-tuning

Tentative definition

Re-train the **whole** model (with or without additional layers) on your data.



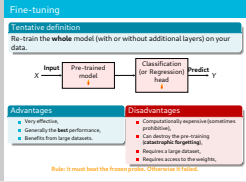
Advantages

- Very effective,
- Generally the **best** performance,
- Benefits from large datasets.

Disadvantages

- Computationally expensive (sometimes prohibitive),
- Can destroy the pre-training (**catastrophic forgetting**),
- Requires a large dataset,
- Requires access to the weights,

Rule: it must beat the frozen probe. Otherwise it failed.



The other end of the spectrum: nothing is frozen, and the pre-trained weights are only an *initialization*.
It usually gives the best score, but suffer from two contradictory problems::

- The **memory**: training needs roughly four times the model size (the weights, their gradients, and two moments per parameter for Adam) plus the activations, so 86 million parameters is comfortable and 7 billion is not.
- The **data**: you are fitting tens of millions of parameters, so overfitting is the default with a few hundred samples.

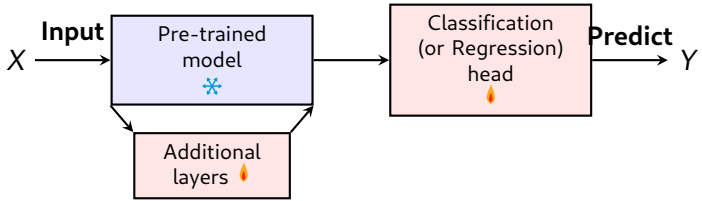
Catastrophic forgetting is a very common problem. Too high a learning rate, and the first gradient steps destroy the knowledge you came for, leaving you *below* the frozen probe. The defences: a very small learning rate (10^{-5} rather than 10^{-3}), few epochs, and freezing the early layers.

Hence the orange line, which should be a reflex.

Parameter-Efficient Fine-Tuning (PEFT)

Tentative definition

Train only a small set of **added parameters**.



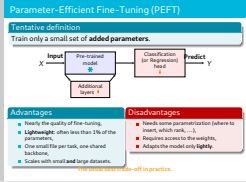
Advantages

- Nearly the quality of fine-tuning,
- **Lightweight**: often less than 1% of the parameters,
- One small file per task, one shared backbone,
- Scales with small **and** large datasets.

Disadvantages

- Needs some parametrization (where to insert, which rank, …),
- Requires access to the weights,
- Adapts the model only **lightly**.

The usual best trade-off in practice.



The middle of the spectrum, and the technique that is used most in practice today, because it takes the good half of each of the two previous slides.

The idea rests on an empirical observation: the *difference* between a pre-trained model and its fine-tuned version is much simpler than the model itself. So instead of letting all the weights move, one freezes them and adds a small number of new parameters that carry the whole adaptation. Since the backbone does not move, catastrophic forgetting is largely avoided (the original knowledge is literally still there), and since the trainable set is tiny, the memory needed for the optimizer collapses.

The third advantage is the one that changed industrial practice, so it is worth picturing. Your adaptation is now a file of a few megabytes rather than a copy of a several-gigabyte model. You can therefore serve one backbone in memory and plug in a different small file per client, per language or per task, and swap them at runtime.

Note that the biggest disadvantage is a list of choices rather than a cost. That is what makes this the default recommendation: the risk is a badly chosen hyperparameter, not a wasted week of compute.

Focus on LoRA (PEFT)

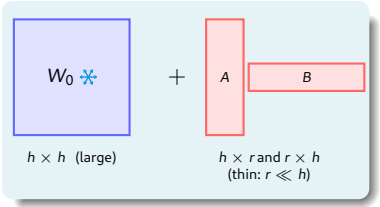
Low-Rank Adaptation

- Freeze the pre-trained matrix W_0
- Learn a **small correction**, written as $A \times B$ ($A \in \mathbb{R}^{h \times r}$ and $B \in \mathbb{R}^{r \times h}$, with r chosen by you ($r = 4, 8, 16, \ldots$).

Pre-trained weights

LoRA weights

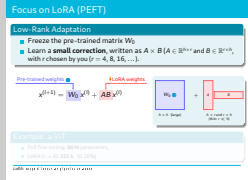
$$x^{(l+1)} = W_0\, x^{(l)} + AB\, x^{(l)}$$



Example: a ViT

- Full fine-tuning: **86 M** parameters,
- LoRA ($r = 4$): **221 k** (0.26%).

LoRA: https://arxiv.org/pdf/2106.09685



The PEFT method you will meet by name everywhere, so it is worth one slide of detail. It is applied mostly to the attention matrices, and it works on MLP layers too.

Read the equation as "the frozen model, plus a correction". During adaptation W_0 never moves; what is learned is the pair (A, B), and the layer computes $W_0 x + ABx$. The trick is in the *shape* of the correction: instead of a full $h \times h$ matrix, it is the product of a tall thin one and a wide flat one, so it has $2hr$ parameters instead of h^2. With $h = 768$ and $r = 4$, that is a factor of about a hundred. The assumption behind it (the intrinsic "rank" of the adaptation is low) is the low-rank idea of session 3 (decomposition methods), reused here.

Two practical bonuses. B is initialized at zero, so at the first step the adapted model is *identical* to the original one, which is a comfortable starting point. And once trained, AB can be added into W_0 once and for all, so the deployed model costs nothing extra at inference.

Focus on LoRA (PEFT)

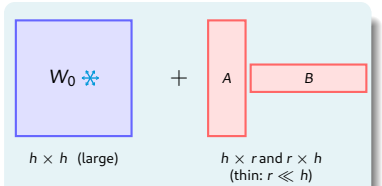
Low-Rank Adaptation

- Freeze the pre-trained matrix W_0
- Learn a **small correction**, written as $A \times B$ ($A \in \mathbb{R}^{h \times r}$ and $B \in \mathbb{R}^{r \times h}$, with r chosen by you ($r = 4, 8, 16, \ldots$).

Pre-trained weights

LoRA weights

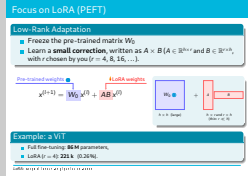
$$x^{(l+1)} = W_0\, x^{(l)} + AB\, x^{(l)}$$



Example: a ViT

- Full fine-tuning: **86 M** parameters,
- LoRA ($r = 4$): **221 k** (0.26%).

LoRA: https://arxiv.org/pdf/2106.09685



The PEFT method you will meet by name everywhere, so it is worth one slide of detail. It is applied mostly to the attention matrices, and it works on MLP layers too.

Read the equation as "the frozen model, plus a correction". During adaptation W_0 never moves; what is learned is the pair (A, B), and the layer computes $W_0 x + ABx$. The trick is in the *shape* of the correction: instead of a full $h \times h$ matrix, it is the product of a tall thin one and a wide flat one, so it has $2hr$ parameters instead of h^2. With $h = 768$ and $r = 4$, that is a factor of about a hundred. The assumption behind it (the intrinsic "rank" of the adaptation is low) is the low-rank idea of session 3 (decomposition methods), reused here.

Two practical bonuses. B is initialized at zero, so at the first step the adapted model is *identical* to the original one, which is a comfortable starting point. And once trained, AB can be added into W_0 once and for all, so the deployed model costs nothing extra at inference.

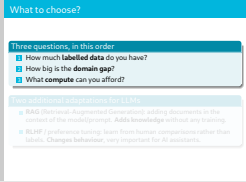
What to choose?

Three questions, in this order

1. How much **labelled data** do you have?
2. How big is the **domain gap**?
3. What **compute** can you afford?

Two additional adaptations for LLMs

- **RAG** (Retrieval-Augmented Generation): adding documents in the context of the model/prompt. **Adds knowledge** without any training.
- **RLHF** / preference tuning: learn from human *comparisons* rather than labels. **Changes behaviour**, very important for AI assistants.



The recap of the section, and the slide to come back to when you have to make the decision for your project.
The three questions are meant to be answered with *numbers*. A few hundred samples and a small domain gap: probe. Thousands, or a real gap: LoRA or the last blocks. Tens of thousands and a GPU: full fine-tuning.

Last block: neither technique changes the weights. **RAG** adds *knowledge* at inference time, encoding your documents into vectors, retrieving those closest to the question, and pasting them into the prompt. **RLHF** changes *behaviour*, from preferences between answers rather than labels.

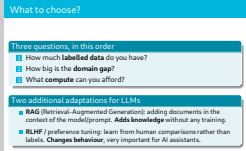
What to choose?

Three questions, in this order

1. How much **labelled data** do you have?
2. How big is the **domain gap**?
3. What **compute** can you afford?

Two additional adaptations for LLMs

- **RAG** (Retrieval-Augmented Generation): adding documents in the context of the model/prompt. **Adds knowledge** without any training.
- **RLHF** / preference tuning: learn from human *comparisons* rather than labels. **Changes behaviour**, very important for AI assistants.



The recap of the section, and the slide to come back to when you have to make the decision for your project.
The three questions are meant to be answered with *numbers*. A few hundred samples and a small domain gap: probe. Thousands, or a real gap: LoRA or the last blocks. Tens of thousands and a GPU: full fine-tuning.

Last block: neither technique changes the weights. **RAG** adds *knowledge* at inference time, encoding your documents into vectors, retrieving those closest to the question, and pasting them into the prompt. **RLHF** changes *behaviour*, from preferences between answers rather than labels.

Outline

1 What is a Foundation Model?

2 Transformers

3 Self-supervised Learning

4 Multimodal Foundation Models

5 Adapting a Foundation Model

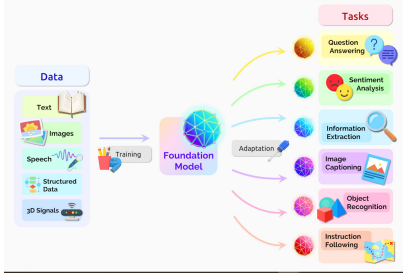
6 Conclusion

Outline
What is a Foundation Model?
Transformers
Self-supervised Learning
Multimodal Foundation Models
Adapting a Foundation Model
Conclusion

What to remember

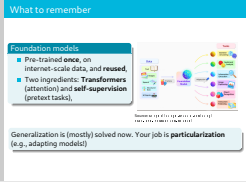
Foundation models

- Pre-trained **once**, on internet-scale data, and **reused**,
- Two ingredients: **Transformers** (attention) and **self-supervision** (pretext tasks),



Source: `https://blogs.nvidia.com/blog/what-are-foundation-models/`

Generalization is (mostly) solved now. Your job is **particularization** (e.g., adapting models!)



Four sentences to take away from this session.
A foundation model is not a better algorithm; it is a **representation paid for by somebody else**, on a quantity of data you will never have. That is the whole reason it works on your problem, and it is also why the honest question about it is always “what has it seen?”.
The two ingredients are worth remembering as a pair, because both explain the power of foundation models: an architecture that accepts any kind of data, and a training signal that needs no annotator. These 2 ingredients are shown to work well with scale, which is the essence of deep learning.
Foundation models lead to a paradigm shift: generalization can be considered solved; the new problem is particularization, i.e., how to adapt a model to your data and your task.
Remember that, because every modality becomes a sequence of vectors, images, text and audio can live in the same model and in the same representation space.

Lab Session 5

Part 1: CIFAR-10 through a **frozen** Vision Transformer

- Load a ViT from **HuggingFace**,
- Extract **embeddings**,
- **Probe** them with supervised learning models (lab 2), and compare with the CNN (lab 4).

Part 2: Your own project data

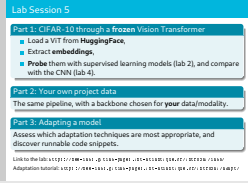
The same pipeline, with a backbone chosen for **your** data/modality.

Part 3: Adapting a model

Assess which adaptation techniques are most appropriate, and discover runnable code snippets.

Link to the lab: `https://mee-labs.gitlab-pages.imt-atlantique.fr/intro2ai/lab5/`

Adaptation tutorial: `https://mee-labs.gitlab-pages.imt-atlantique.fr/intro2ai/adapt/`



The lab should be shorter than the previous ones: reusing a model pre-trained on fourteen million images replaces most of the work of lab 4. Use the time you save on your project.

Part 1 introduces you to the use of foundation models for vision (in Transfer Learning), and you should compare its performance against baselines: chance level, raw pixels, your lab-4 CNN, all on the *same* test split. You will likely train several thousand parameters for a few seconds and beat the network you trained for dozens of epochs. Make sure you can say why in one sentence.

Part 2 matters for your project, and you should estimate the **domain gap**: a ViT pre-trained on internet photographs is close to your holiday pictures and far from an electron micrograph. A negative result is a good result here, if you can diagnose it.

Part 3 gives you a first sight on how you could adapt a model for your data.